

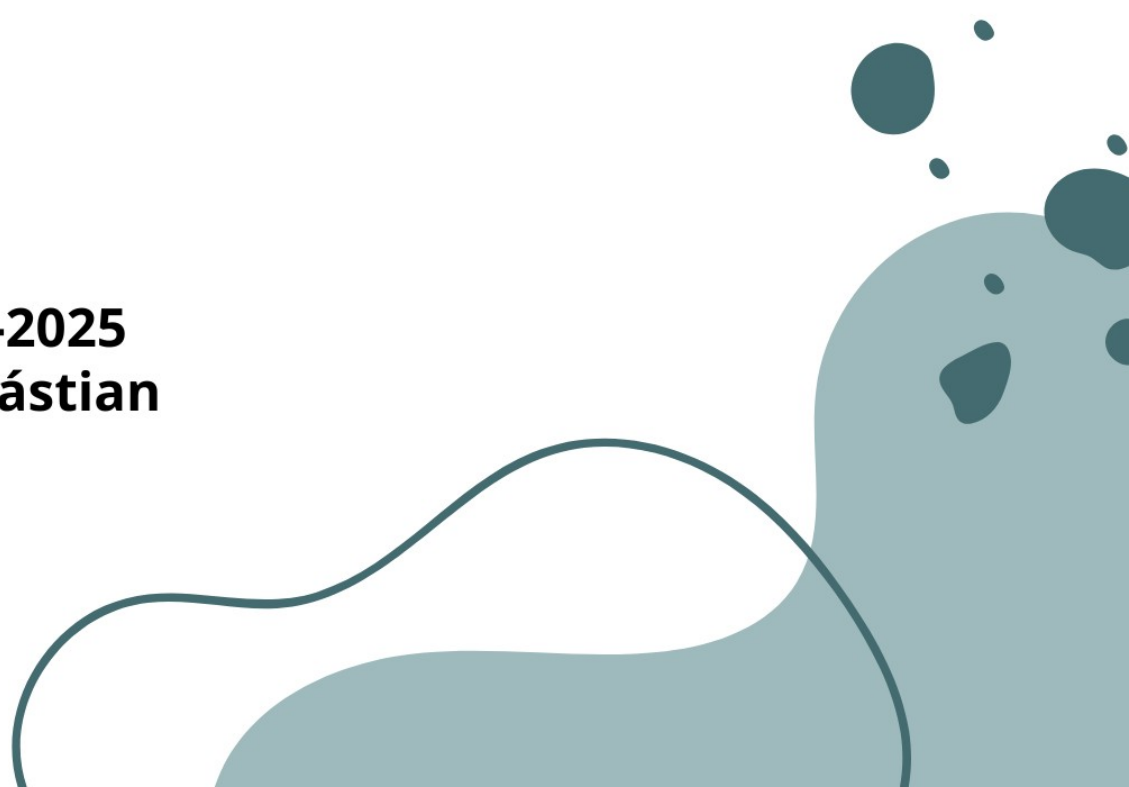


Desarrollo Web Entorno Servidor

MEMORIA

Hecho por: Roy Lee Lunar Moré

**2º DAW
Curso 2024-2025
IES San Sebastián**



INDICE

Migraciones, Factorías y Modelos:.....	2
Usuarios, login y permisos:.....	8
Hilos de Conversación y posts:.....	15
Ranking:.....	20
Admin:.....	22
Imagen y Rutas:.....	25

Migraciones, Factorías y Modelos:

Para realizar la actividad es necesario crear 4 tablas:

Tabla Users: Esta tabla posee los atributos id, name, email, password e img_perfil (contiene la ruta de la imagen de perfil en el servidor). La tabla tiene una relación 1 a N con la tabla Posts, Likes e Hilos.

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name', 60);
    $table->string('email', 100);
    $table->string('password');
    $table->string('img_perfil');
    $table->timestamps();
});
```

Tabla Hilos: Esta tabla posee los atributos id, titulo, mensaje y la clave foránea user_id que hace referencia al creador del hilo. Esta tabla posee una relación de N a 1 con la tabla Users y una relación de 1 a N con la tabla Posts.

```
Schema::create('hilos', function (Blueprint $table) {
    $table->id();
    $table->string('titulo', 320);
    $table->string('mensaje', 320);
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->timestamps();
});
```

Tabla Posts: Esta tabla posee los atributos id, mensaje y las claves foráneas hilo_id y user_id que hacen referencia de quien es y de que hilo pertenece este post. Esta tabla tiene un relación de 1 a N con la tabla Likes, además posee una relación de N a 1 con las tablas Hilos y Users.

```
Schema::create('posts', function (Blueprint $table) {
    $table->id();
    $table->string('mensaje', 350);
    $table->foreignId('hilo_id')->constrained()->onDelete('cascade');
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->timestamps();
});
```

Tabla Like: Esta tabla solo posee los atributos id y las claves foráneas user_id y post_id que hacen referencia de quien dio el like y a que post lo dio, user_id y post_id debemos declararlos como únicos para que un usuario no pueda dar mas de un like a un post. Esta tabla tiene una relación de N a 1 con las tablas Users y Posts.

```
Schema::create('hilos', function (Blueprint $table) {
    $table->id();
    $table->string('titulo', 320);
    $table->string('mensaje', 320);
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->timestamps();
});
```

Para ejecutar las migraciones y que nos cree nuestras tablas en la base de datos debemos usar el comando 'php artisan migrate'.

```
PS C:\Users\Linkm\OneDrive\Documentos\1 - Grado Superior\1 - 2º DAW\ - Desarrollo Web Entorno Servidor\Unidad 8\Tarea\T8_A1_L
unarMore_Roy> php artisan migrate
Xdebug: [Step Debug] Time-out connecting to debugging client, waited: 200 ms. Tried: localhost:9003 (through xdebug.client_
host/xdebug.client_port).

[INFO] Preparing database.

Creating migration table ..... 70.75ms DONE

[INFO] Running migrations.

0001_01_01_000000_create_users_table ..... 44.05ms DONE
0001_01_01_000001_create_cache_table ..... 10.76ms DONE
0001_01_01_000002_create_jobs_table ..... 56.36ms DONE
2025_02_12_151045_create_hilos_table ..... 41.91ms DONE
2025_02_12_151049_create_posts_table ..... 59.30ms DONE
2025_02_12_152513_create_likes_table ..... 73.56ms DONE
2025_02_12_153305_create_permission_tables ..... 183.69ms DONE

Xdebug: [Step Debug] Time-out connecting to debugging client, waited: 200 ms. Tried: localhost:9003 (through xdebug.client_
host/xdebug.client_port).
PS C:\Users\Linkm\OneDrive\Documentos\1 - Grado Superior\1 - 2º DAW\ - Desarrollo Web Entorno Servidor\Unidad 8\Tarea\T8_A1_L
unarMore_Roy>
```

Si nos dirigimos ahora a la base de datos podemos ver que se crearon exitosamente todas las tablas.

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐ cache	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ cache_locks	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ failed_jobs	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ hilos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ jobs	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ job_batches	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ likes	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	64.0 KB	-
☐ migrations	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	7	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ model_has_permissions	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ model_has_roles	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ password_reset_tokens	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ permissions	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ posts	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
☐ roles	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ role_has_permissions	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ sessions	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
☐ users	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
17 tablas	Número de filas	7	InnoDB	utf8mb4_general_ci	512.0 KB	0 B

☐ Seleccionar todo
 Para los elementos que están marcados:

Los modelos para nuestras tablas los definiremos de la siguiente forma con sus relaciones para sus tablas específicas.

Aquí podemos ver como he definido los métodos y relaciones entre las tablas tanto las normales como las polimorficas.

```
class Hilo extends Model
{
    use HasFactory;
    protected $fillable = ['titulo', 'mensaje', 'user_id'];

    public function user() {
        return $this->belongsTo(User::class);
    }

    public function post() {
        return $this->hasMany(Post::class);
    }
}
```

```
class Like extends Model
{
    use HasFactory;
    protected $fillable = ['user_id', 'post_id'];

    public function post(){
        return $this->belongsTo(Post::class);
    }

    public function user(){
        return $this->belongsTo(User::class);
    }
}
```

```
class Post extends Model
{
    use HasFactory;

    protected $fillable = ['mensaje', 'user_id', 'hilo_id'];

    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function hilo()
    {
        return $this->belongsTo(Hilo::class);
    }

    public function likes()
    {
        return $this->hasMany(Like::class);
    }
}
```

```
class User extends Authenticatable
{
    public function like() {
        return $this->hasMany(Like::class);
    }

    public function post() {
        return $this->hasMany(Post::class);
    }

    public function hilo() {
        return $this->hasMany(Hilo::class);
    }
}
```

Las factorías para estas tablas son muy sencilla creamos datos de forma aleatoria con faker. Por ejemplo para la factoría de Hilo tomaremos un usuario de forma aleatoria para crear un hilo.

```
class HiloFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'titulo' => $this->faker->sentence(6),
            'mensaje' => $this->faker->sentence(20),
            'user_id' => User::inRandomOrder()->first()->id,
        ];
    }
}
```

Haremos lo mismo para la factoría de Post solo que en este caso tomaremos de forma aleatoria el id de un hilo para asignárselo al post.

```
class PostFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'mensaje' => $this->faker->sentence(18),
            'hilo_id' => Hilo::inRandomOrder()->first()->id,
            'user_id' => User::inRandomOrder()->first()->id,
        ];
    }
}
```

La factoría de Likes es un poco más complicada porque como necesitamos crear likes únicos evitando duplicados debemos especificarle a la factoría que combinaciones ya ha hecho para que no vuelva a hacerla y no de un error.

Tomaremos el id de todos los posts y usuarios y tomaremos de forma aleatoria uno de cada uno con un bucle do while y comprobaremos que no se encuentre en nuestro array de combinaciones que ya haya realizado y si esa combinación no existe la registramos y creamos el like.

```

class LikeFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    protected static $usedCombinations = [];

    public function definition(): array
    {
        $userIds = User::pluck('id')->toArray();
        $postIds = Post::pluck('id')->toArray();

        // Si no hay usuarios o posts, lanzar una excepción
        if (empty($userIds) || empty($postIds)) {
            throw new \Exception("No hay usuarios o posts en la base de datos.");
        }

        do {
            $user_id = $this->faker->randomElement($userIds);
            $post_id = $this->faker->randomElement($postIds);
            $key = "{$user_id}-{$post_id}"; // Clave única para la combinación
        } while (
            in_array($key, self::$usedCombinations) ||
            Like::where('user_id', $user_id)->where('post_id', $post_id)->exists()
        );

        // Registrar la combinación para que no se repita en esta ejecución
        self::$usedCombinations[] = $key;

        return [
            'user_id' => $user_id,
            'post_id' => $post_id
        ];
    }
}

```

Para la factoría de usuario usamos la que trae por defecto laravel y aplicamos los cambios para añadir el campo `img_perfil`, una vez hecho tomaremos de forma aleatoria con `faker` una de las 5 imágenes por defecto para los usuarios. Por ultimo definimos una función `configure` que hace que cuando cree dicho usuario podamos asignarle de forma aleatoria un rol.

```

class UserFactory extends Factory
{
    /**...
    // protected static ?string $password;

    /**...
    public function definition(): array
    {
        return [
            'name' => $this->faker->name(),
            'email' => $this->faker->email(),
            'password' => $this->faker->password(),
            'img_perfil' => $this->faker->randomElement(['avatar1.png', 'avatar2.png',
                                                    'avatar3.png', 'avatar4.png',
                                                    'avatar5.png']),
        ];
    }

    /**...
    public function unverified(): static
    {
        return $this->state(fn(array $attributes) => [
            'email_verified_at' => null,
        ]);
    }

    public function configure()
    {
        return $this->afterCreating(function (User $user) {
            $roles = ['admin', 'editor', 'user'];
            $user->assignRole($this->faker->randomElement($roles));
        });
    }
}

```

En mi proyecto hago uso del seeder DatabaseSeeder, este se encarga de llamar de forma automática las factorías de nuestras tablas indicando de forma repetida las veces que deseemos a su vez también lo uso para definir los permisos y asignarlos a los roles user, editor y admin.

```

public function run(): void
{
    Permission::create(['name' => 'admin.edit.rol']);
    Permission::create(['name' => 'admin.del.user']);
    Permission::create(['name' => 'admin.del.post']);
    Permission::create(['name' => 'ver.users']);
    Permission::create(['name' => 'edit.user']);
    Permission::create(['name' => 'user.add.post']);
    Permission::create(['name' => 'user.add.hilo']);

    // Crear roles y asignar permisos
    $adminRole = Role::create(['name' => 'admin']);
    $adminRole->givePermissionTo(Permission::all()); // Asigna todos los permisos al rol admin

    $editorRole = Role::create(['name' => 'editor']);
    $editorRole->givePermissionTo(['ver.users', 'edit.user', 'user.add.post', 'user.add.hilo']);

    $usuarioRole = Role::create(['name' => 'user']);
    $usuarioRole->givePermissionTo(['user.add.post', 'user.add.hilo']);

    User::factory()->count(20)->create();
    Hilo::factory()->count(5)->create();
    Post::factory()->count(30)->create();
    Like::factory()->count(40)->create();
}

```


Para poder realizar la inserción de base de datos hago uso del comando '[php artisan db:seed](#)'.

```
PS C:\Users\Linkm\OneDrive\Documentos\1 - Grado Superior\1 - 2º DAW\ - Desarrollo Web Entorno Servidor\Unidad 8\Tarea\T8_A1_L
• unarMore_Roy> php artisan db:seed
Xdebug: [Step Debug] Time-out connecting to debugging client, waited: 200 ms. Tried: localhost:9003 (through xdebug.client_
host/xdebug.client_port).

INFO Seeding database.

Xdebug: [Step Debug] Time-out connecting to debugging client, waited: 200 ms. Tried: localhost:9003 (through xdebug.client_
host/xdebug.client_port).
Xdebug: [Step Debug] Time-out connecting to debugging client, waited: 200 ms. Tried: localhost:9003 (through xdebug.client_
host/xdebug.client_port).
PS C:\Users\Linkm\OneDrive\Documentos\1 - Grado Superior\1 - 2º DAW\ - Desarrollo Web Entorno Servidor\Unidad 8\Tarea\T8_A1_L
unarMore_Roy>
```

Usuarios, login y permisos:

Para el login y el registro use el que trae por defecto Laravel, los cambios que tuve que realizar fueron mínimos, tuve que cambiar el LoginRequest para que aceptara los nombres porque anteriormente realizaba el login con el correo que tuviera el usuario.

En el LoginRequest como podemos ver cambiamos correo por nombre.

```
class LoginRequest extends FormRequest
{
    *
    * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
    */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'min:3', 'max:20'],
            'password' => ['required', 'string'],
        ];
    }

    public function messages(){
        return [
            'name.required' => 'El nombre es obligatorio.',
            'name.string' => 'El nombre debe ser una cadena de texto.',
            'name.min' => 'El nombre debe tener al menos 3 caracteres.',
            'name.max' => 'El nombre no puede tener más de 20 caracteres.',
            'password.required' => 'La contraseña es obligatoria.',
            'password.string' => 'La contraseña debe ser una cadena de texto.',
        ];
    }
}
```

Para cada regla rota definimos sus mensajes en español para guiar al usuario. El resultado en la web sería el siguiente.

The image shows a login form on a dark blue background. At the top, there is a button labeled 'Iniciar sesión'. Below it, the form contains two input fields: 'Nombre' (Name) and 'Contraseña' (Password). Under the password field, there is a checkbox labeled 'Recuérdame' (Remember me). At the bottom of the form, there are two links: 'Olvidaste tu contraseña?' (Forgot your password?) and 'No tienes cuenta?' (Don't have an account?). To the right of these links is another button labeled 'INICIAR SESIÓN'.

En el registro añadí un campo extra que es el input file para subir la imagen y edite el RegisterRequest que trae para que verifique que tanto el correo como el nombre que ingreso el usuario para el registro ya existe, luego añadí una regla extra para el campo img_perfil para indicar que el campo es obligatorio y que debe ser una imagen.

```
class RegisterRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'min:3', 'max:20', 'unique:' . User::class],
            'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:' . User::class],
            'img_perfil' => [
                'required',
                File::image()
            ],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ];
    }
}
```

A su vez también agregué los mensajes personalizados de error en español. Este sería el resultado final de nuestro registrarse.

Create una cuenta

Nombre

Correo Electronico

Imagen

Seleccionar archivo

Contraseña

Confirmar Contraseña

[Ya te registraste?](#)

REGISTRATE

Cuando el usuario se haya registrado o iniciado sesión se le mostrará su nombre, foto de perfil y rol en la barra de navegación esto lo hice modificando el componente **navigation** que trae Laravel para que muestre la imagen y el rol junto al nombre de usuario.

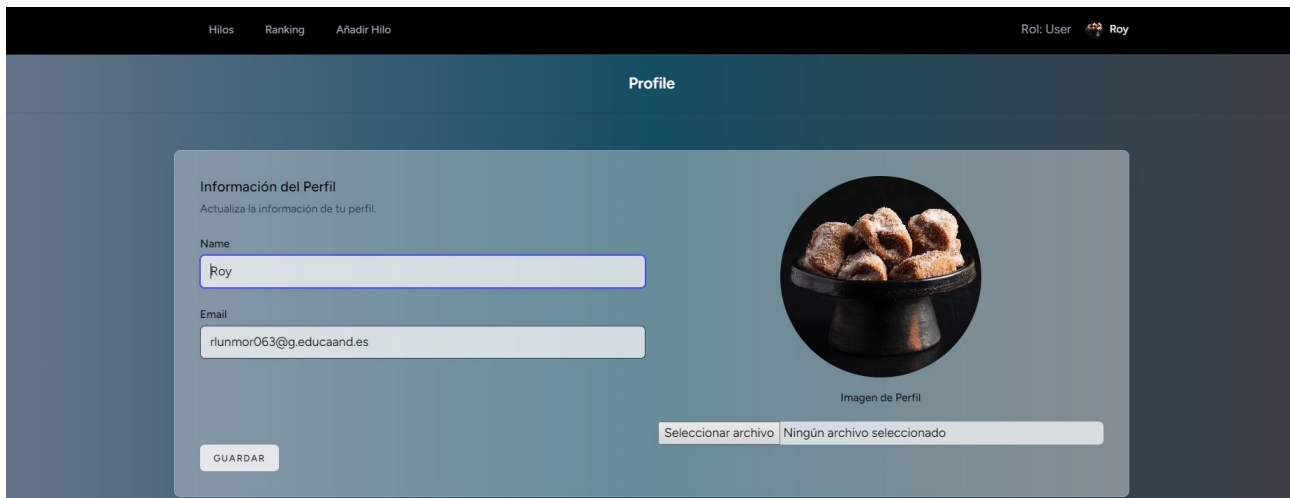
```
@if (Route::has('login'))
<nav class="mx-3 flex flex-1 justify-end">
  <div class="text-gray-400 dark:text-gray-400 flex items-center">Rol:
    {{ ucfirst(Auth::user()->getRoleNames()->first()) }}</div>
  <button
    class="inline-flex items-center px-3 py-2 border border-transparent text-sm leading-4 font-medium rounded-md text-gray-500 dark:text-gray-600"
    >
    <div class="ms-1 flex items-center">
      
    </div>
    <div class="ml-2 dark:text-gray-100">{{ Auth::user()->name }}</div>
  </button>
</nav>
@endif
```

En la vista se vería de la siguiente forma:



El panel de control para que el usuario modifique sus datos es el mismo que trae por defecto Laravel, le aplique unos cuantos cambios y la traducción al español. Agregué un

espacio para que el usuario pudiera cambiar su imagen de perfil y pudiera ver la imagen actual que llevaba.



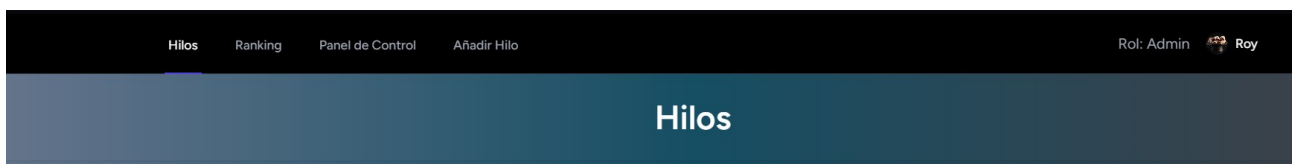
Si cambiamos un datos en este caso el correo y guardamos los datos podremos ver como en la base de datos se actualiza.

☐	 Editar	 Copiar	 Borrar	19	Nakia Yost	aanderson@hickle.com	\$2y\$12\$clfv7YFNupwV4bCE077eLMSGkms4BowDr1ieEO4gs...	avatar3.png	2025-02-25 15:09:08
☐	 Editar	 Copiar	 Borrar	20	Bennie Bogan	karina53@yahoo.com	\$2y\$12\$Jl4hh.NpaJSYKPMCaXIO2OZ6vLHh3iT1wsL2xjkyP...	avatar3.png	2025-02-25 15:09:08
☐	 Editar	 Copiar	 Borrar	21	Roy	roymodificado063@g.educaand.es	\$2y\$12\$ZA7YPGIMKzq4KJskAgfSQOcV93Oj0Mjr48MnPTKfDQn...	ICcpgpdwoKr9KYCN3SkVHD0DATth8ABoVMTsuWZh.jpg	2025-02-25 15:27:57

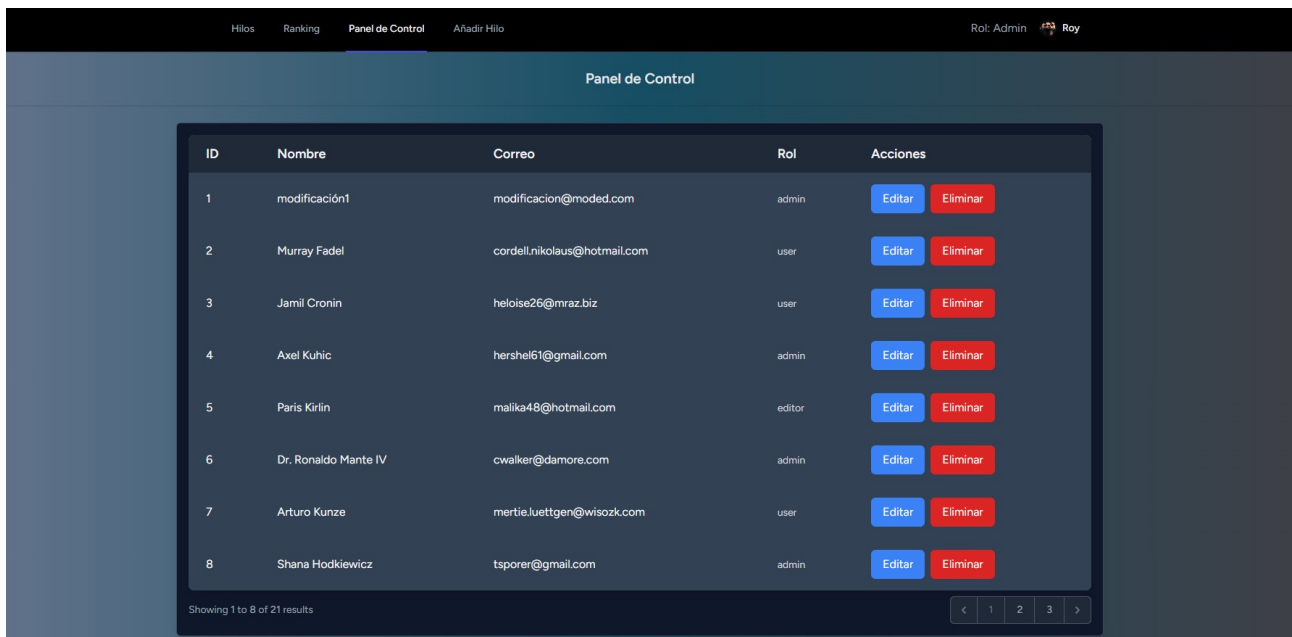
Haciendo uso de Spatie logro mostrar el botón adicional para ver todos los usuarios poder editarlos y eliminar el que desee. Esto lo hago haciendo uso de @can que trae blade en donde recibe por parámetro el permiso que necesita dicho usuario para ver el contenido que se encuentre dentro del bloque.

```
<!-- Navigation Links -->
<div class="hidden space-x-8 sm:-my-px sm:ms-10 sm:flex">
  <x-nav-link :href="route('home')" :active="request()->routeIs('home')">
    {{ __('Hilos') }}
  </x-nav-link>
  <x-nav-link :href="route('ranking')" :active="request()->routeIs('ranking')">
    {{ __('Ranking') }}
  </x-nav-link>
  @can('ver.users')
    <x-nav-link :href="route('ver.users')" :active="request()->routeIs('ver.users')">
      {{ __('Panel de Control') }}
    </x-nav-link>
  @endcan
  @can('user.add.hilo')
    <x-nav-link :href="route('user.add.hilo')" :active="request()->routeIs('user.add.hilo')">
      {{ __('Añadir Hilo') }}
    </x-nav-link>
  @endcan
</div>
```

Si tenemos el rol de administrado podemos ver que el botón aparece en el menú de navegación.



Si nos dirigimos ahí veremos los registro y los datos de cada usuario junta a las acciones de editar y eliminar.



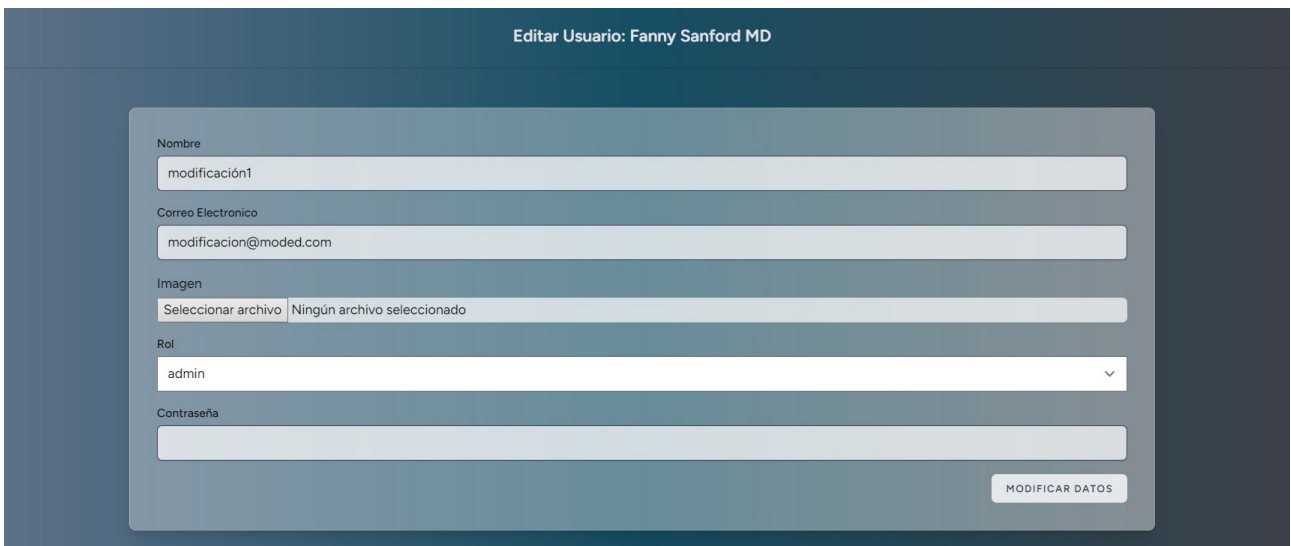
Para que los usuarios usuarios editores también puedan acceder a los registros deben también tener el permiso para verlos pero no pueden eliminarlos esto lo hago nuevamente controlando con el @can en donde compruebo si el usuario tiene el permiso de eliminar el usuario para mostrarle el botón de eliminar o no.

```
@can('admin.del.user')
<form action="{{ route('admin.del.user', $user) }}" method="POST" class="inline">
  @csrf
  @method('DELETE')
  <button type="submit"
    class="px-4 py-2 rounded-md bg-red-600 text-white hover:bg-red-800 transition duration-300">
    Eliminar
  </button>
</form>
@endcan
```

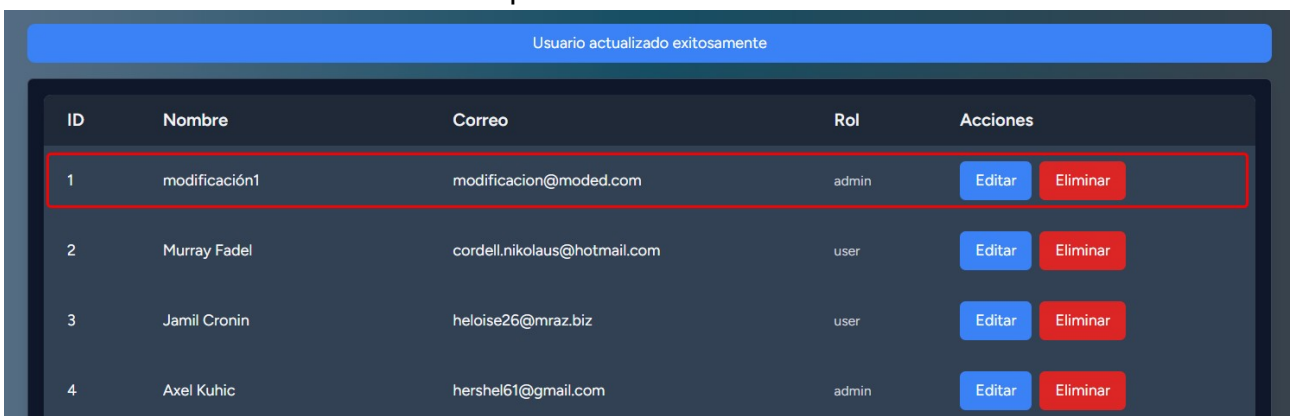
Y para no permitir que el editor pueda cambiar el rol utilizo el mismo método, compruebo si el usuario tiene el permiso para editar el rol y si lo tiene le muestro el input para cambiar el rol.

```
@can('admin.edit.rol')
<div class="mt-4">
  <x-input-label for="rol" :value="__('Rol')"/>
  <select name="rol" id="rol" class="block mt-1 w-full">
    @foreach ($roles as $rol)
      <option value="{{ $rol }}" {{ $user->hasRole($rol) ? 'selected' : '' }}>
        {{ $rol }}</option>
    @endforeach
  </select>
  <x-input-error :messages="$errors->get('rol')" class="mt-2"/>
</div>
@endcan
```

Y podremos ver que a la hora de modificar tenemos habilitado el rol.



Si modificamos los datos veremos que se actualiza correctamente.



ID	Nombre	Correo	Rol	Acciones
1	modificación1	modificacion@moded.com	admin	<button>Editar</button> <button>Eliminar</button>
2	Murray Fadel	cordell.nikolaus@hotmail.com	user	<button>Editar</button> <button>Eliminar</button>
3	Jamil Cronin	heloise26@mraz.biz	user	<button>Editar</button> <button>Eliminar</button>
4	Axel Kuhic	hershel61@gmail.com	admin	<button>Editar</button> <button>Eliminar</button>

Si nos dirigimos a la base de datos podemos ver que los datos se actualizaron de forma correcta.

Mostrando filas 0 - 20 (total de 21, La consulta tardó 0,0003 segundos)

SELECT * FROM `users`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	id	name	email	password	img_perfil
☐ Editar ☐ Copiar ☐ Borrar	1	modificación1	modificacion@moded.com	\$2y\$12\$zsJXT02.MPMU45yHbKU6eo6lCV3X8b7Lsj6gF31eAn...	avatar4.png
☐ Editar ☐ Copiar ☐ Borrar	2	Murray Fadel	cordell.nikolaus@hotmail.com	\$2y\$12\$4pdKWZdX0V6oQ/fNvtpQse4dGz.E1O2Js83qQCK8g4x...	avatar2.png
☐ Editar ☐ Copiar ☐ Borrar	3	Jamil Cronin	heloise26@mraz.biz	\$2y\$12\$LBV776Njo6ovSsPR9nE9g.xVO.gEXcpobBmKFA6KBcn...	avatar3.png

Pero si entramos como usuario notaremos que todos la opciones tanto de eliminar como cambiar el rol ya no existen.

Hilos Ranking Panel de Control Añadir Hilo Rol: Editor Roy

Panel de Control

Usuario actualizado exitosamente

ID	Nombre	Correo	Rol	Acciones
1	modificación1	modificacion@moded.com	admin	Editar
2	Murray Fadel	cordell.nikolaus@hotmail.com	user	Editar
3	Jamil Cronin	heloise26@mraz.biz	user	Editar
4	Axel Kuhic	hershel61@gmail.com	admin	Editar

Ya no tendremos acceso a modificar el usuario

Hilos Ranking Panel de Control Añadir Hilo Rol: Editor Roy

Editar Usuario: modificación1

Nombre
modificación1

Correo Electronico
modificacion@moded.com

Imagen
Seleccionar archivo Ningún archivo seleccionado

Contraseña

También hay que controlar en el FormRequest si el rol de usuario es admin para agregar las reglas de validación del rol y comprobamos también si los datos específicos del campo correo y nombre no existe para poder modificarlo.

```

public function rules(): array
{
    $userId = $this->route('user'); // Asumiendo que el ID del usuario se pasa a través de la ruta

    return [
        'name' => ['required', 'string', 'min:3', 'max:20', 'unique:users,name,' . $userId],
        'email' => ['required', 'string', 'email', 'max:255', 'unique:users,email,' . $userId],
        'img_perfil' => ['image', 'mimes:jpeg,png,jpg,webp', 'max:10240'],
        'password' => ['nullable', Rules\Password::defaults()],
    ];

    if (auth()->user->hasRole('admin')) {
        $rules['rol'] = ['required', 'string', 'in:admin,editor,user'];
    }
}

```

A su vez en este FormRequest definimos los mensajes de error personalizados en español.

```

class EditRequest extends FormRequest
{
    public function messages()
    {
        return [
            'name.required' => 'El campo nombre es obligatorio.',
            'name.min' => 'El campo nombre debe tener al menos 3 caracteres.',
            'name.max' => 'El campo nombre no puede tener más de 20 caracteres.',
            'name.string' => 'El campo nombre debe ser una cadena de texto.',
            'name.unique' => 'Ya existe un con este nombre.',
            'email.email' => 'El campo correo debe ser una dirección de correo electrónico válida.',
            'email.lowercase' => 'El campo correo debe estar en minúsculas.',
            'email.max' => 'El correo no puede tener más de 255 caracteres.',
            'email.string' => 'El campo correo debe ser una cadena de texto.',
            'email.unique' => 'Ya existe un usuarios con este correo',
            'email.required' => 'El campo correo es obligatorio.',
            'img_perfil.image' => 'El archivo debe ser una imagen.',
            'img_perfil.max' => 'El archivo no puede tener más de 10MB.',
            'img_perfil.mimes' => 'El archivo debe tener una extensión válida (jpeg, png, jpg, webp).',
            'img_perfil.nullable' => 'La imagen de perfil es opcional.',
            'password.required' => 'La contraseña es obligatorio.',
            'password.min' => 'La contraseña debe tener al menos 8 caracteres.',
            'password.max' => 'La contraseña no puede tener más de 255 caracteres.',
            'password.string' => 'La contraseña debe ser una cadena de texto.',
            'rol.in' => 'El rol debe ser admin, editor o user.',
            'rol.string' => 'El rol debe ser una cadena de texto.',
        ];
    }
}

```

Hilos de Conversación y posts:

Para crear hilos nuevos añadimos una nueva opción en el menú de navegación llamada Añadir Hilo esta te redirige a otra ruta con un formulario con su propio form request para validar los campos. Esta opción solo aparece si el usuario tiene los permisos user.add.hilo con el bloque @can de blade.


```

<!-- Navigation Links -->
<div class="hidden space-x-8 sm:-my-px sm:ms-10 sm:flex">
  <x-nav-link :href="route('home')" :active="request()->routeIs('home')">
    {{ __('Hilos') }}
  </x-nav-link>
  <x-nav-link :href="route('ranking')" :active="request()->routeIs('ranking')">
    {{ __('Ranking') }}
  </x-nav-link>
  @can('ver.users')
    <x-nav-link :href="route('ver.users')" :active="request()->routeIs('ver.users')">
      {{ __('Panel de Control') }}
    </x-nav-link>
  @endcan
  @can('user.add.hilo')
    <x-nav-link :href="route('user.add.hilo')" :active="request()->routeIs('user.add.hilo')">
      {{ __('Añadir Hilo') }}
    </x-nav-link>
  @endcan
</div>

```

Este permiso lo tienen todos los usuarios que tengan roles en nuestro proyecto o sea los usuarios no registrados no tendrán acceso a crear un hilo nuevo.

Así lucirá nuestro formulario para añadir un nuevo hilo.

Si intentamos crear un nuevo hilo añadiremos un nuevo hilo a nuestra base de datos y el FormRequest se encargará de validar los campos y enviar mensajes personalizados a los usuarios.

```

class HiloRequest extends FormRequest
{
    /**
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'hiloname' => ['required', 'string', 'max:100', 'min:3'],
            'hilomsg' => ['required', 'string', 'max:255', 'min:3'],
        ];
    }
}

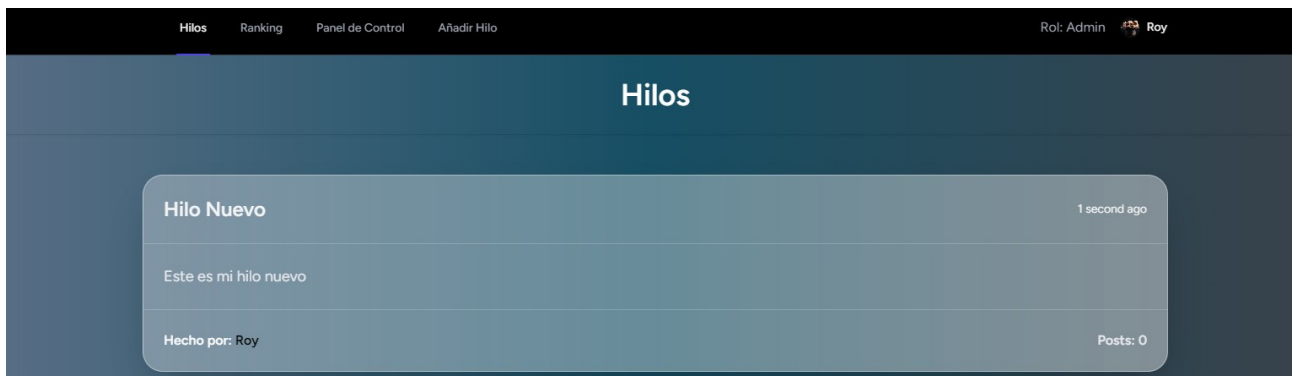
```

```

public function messages()
{
    return [
        'hiloname.required' => 'El título de hilo es obligatorio.',
        'hiloname.string' => 'El título debe ser una cadena de texto.',
        'hiloname.max' => 'El título no puede tener más de 100 caracteres.',
        'hiloname.min' => 'El título debe tener al menos 3 caracteres.',
        'hilomsg.required' => 'El contenido del hilo es obligatorio.',
        'hilomsg.string' => 'El mensaje debe ser una cadena de texto.',
        'hilomsg.max' => 'El mensaje no puede tener más de 255 caracteres.',
        'hilomsg.min' => 'El mensaje debe tener al menos 3 caracteres.',
    ];
}
}

```

Y al enviar nuestro nuevo hilo se creara correctamente.



Veremos en la base de datos que se creo el contenido exitosamente

		id	titulo	mensaje	user_id	created_at	updated_at	
			1	Et ut doloreque omnis officia quam beatae quis.	Nihil rerum et qui occaecati qui temporibus laboru...	17	2025-02-25 15:09:08	2025-02-25 15:09:08
			2	In numquam ut perferendis minima est quasi neque v...	Autem laboriosam velit pariatur eaque quod non qui...	10	2025-02-25 15:09:08	2025-02-25 15:09:08
			3	Rem doloreque dicta voluptatem eveniet ut deserun...	Dolores tempore expedita illo nihil non consequatu...	13	2025-02-25 15:09:08	2025-02-25 15:09:08
			4	Ea nemo velit sit neque quia officiis corporis rem...	Molestiae et amet non aut voluptatem voluptas ipsa...	11	2025-02-25 15:09:08	2025-02-25 15:09:08
			5	Eos ut ad sit reprehenderit.	Blanditis sed magni omnis ex nostrum rem ut excep...	5	2025-02-25 15:09:08	2025-02-25 15:09:08
			6	Hilo Nuevo	Este es mi hilo nuevo	21	2025-02-25 17:47:05	2025-02-25 17:47:05

Los hilos se listan ordenados de forma ascendente por fecha siendo los mas nuevos los primeros en mostrarse, estos son un enlace que te redirige al contenido de dicho hilo y se muestran los posts asociados a dicho hilo. Si accedemos a dicho hilo podemos ver todos los posts de dicho hilo ordenados por fecha de forma descendente.

[Hilos](#) [Ranking](#) [Panel de Control](#) [Añadir Hilo](#) Rol: Admin  Roy

Hilo Nuevo 9 minutes ago

Este es mi hilo nuevo

Hecho por: Roy

Este hilo no tiene posts aún

Mensaje

CREAR POST

En este caso en la parte final de los posts podemos crear uno nuevo este es un formulario que crea un nuevo post y lo añade a este hilo, este también tiene su propio FormRequest con mensajes en español y reglas para validar los campos.

```
class PostRequest extends FormRequest
{
    /** ...
    public function rules(): array
    {
        return [
            'postmsg' => ['required', 'string', 'max:255', 'min:3'],
        ];
    }

    public function messages()
    {
        return [
            'postmsg.required' => 'El contenido del hilo es obligatorio.',
            'postmsg.string' => 'El mensaje debe ser una cadena de texto.',
            'postmsg.max' => 'El mensaje no puede tener más de 255 caracteres.',
            'postmsg.min' => 'El mensaje debe tener al menos 3 caracteres.',
        ];
    }
}
```

Cuando añadimos el post nuevo mostrará la cantidad de likes que tiene el post el contenido, su autor junto a su imagen de perfil y hace cuanto tiempo se hizo el post junto a un botón de eliminar si eres admin.

Crearemos un post nuevo y comprobaremos la base de datos.

Para mostrar el corazón relleno o vacío comprobamos con un bloque @if una consulta hacia la base de datos comprobando si el usuario dio like o no.

```
<form action="{{ route('user.like.post', $post) }}" class="items-center justify-center"
method="POST">
@csrf
<button type="submit"
class="text-red-500 hover:scale-110 transition flex justify-center items-center">
@if ($post->likes()->where('user_id', auth()->id())->exists())
<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24"
fill="red" viewBox="0 0 24 24">
<path
d="M12 21.351-1.45-1.32C5.4 15.36 2 12.28 2 8.5 2 5.42 4.42 3 7.5 3c1.74 0 3.41.81 4.5 2.09C13.09 3
</svg>
@else
<svg xmlns="http://www.w3.org/2000/svg" width="24" height="24"
viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2">
<path
d="M12 21.351-1.45-1.32C5.4 15.36 2 12.28 2 8.5 2 5.42 4.42 3 7.5 3c1.74 0 3.41.81 4.5 2.09C13.09 3
</svg>
@endif
</button>
</form>
```

Y en el controlador comprobamos si existe o no este like para eliminarlo o crearlo.

```
public function like($post){
    $user = auth()->guard('web')->user();

    $existeLike = Like::where('user_id', $user->id)->where('post_id', $post)->first();

    if ($existeLike) {
        $existeLike->delete();
    } else {
        $post = Post::findOrFail($post);
        $post->likes()->create([
            'user_id' => $user->id,
        ]);
    }

    return redirect()->back();
}
```

Ranking:

Para consultar los usuarios que han recibido mas likes hacemos una consulta donde contamos los post como likes totales y con un join tomamos el id de la tabla post y lo comparamos con el post_id de la tabla de likes, luego los ordenamos de forma descendente y tomamos los 10 primero para que sea un top 10 de usuarios que han recibido mas likes.

```

public function ranking()
{
    $top = User::withCount(['post as total_likes' => function ($query) {
        $query->join('likes', 'posts.id', '=', 'likes.post_id');
    }])->orderByDesc('total_likes')->take(10)->get();

    return view('control.ranking')->with('top', $top);
}

```

En la vista lucirá de la siguiente forma:



Cuando la vista reciba la información recorreremos cada usuario e imprimiremos su posición su nombre y el total de likes que ha recibido el usuario.

```

<ol class="space-y-4">
    @isset($top)
        @foreach ($top as $index => $user)
            <li class="p-4 bg-white dark:bg-gray-700 rounded-lg shadow hover:bg-gray-50 dark:hover:bg-gray-600">
                <p class="font-semibold text-xl text-gray-800 dark:text-gray-200 leading-tight">
                    <span class="text-blue-500">{{ $index + 1 }}</span> {{ $user->name }} -
                    <span class="text-yellow-500">{{ $user->total_likes }} me gusta</span>
                </p>
            </li>
        @endforeach
    @endisset
</ol>

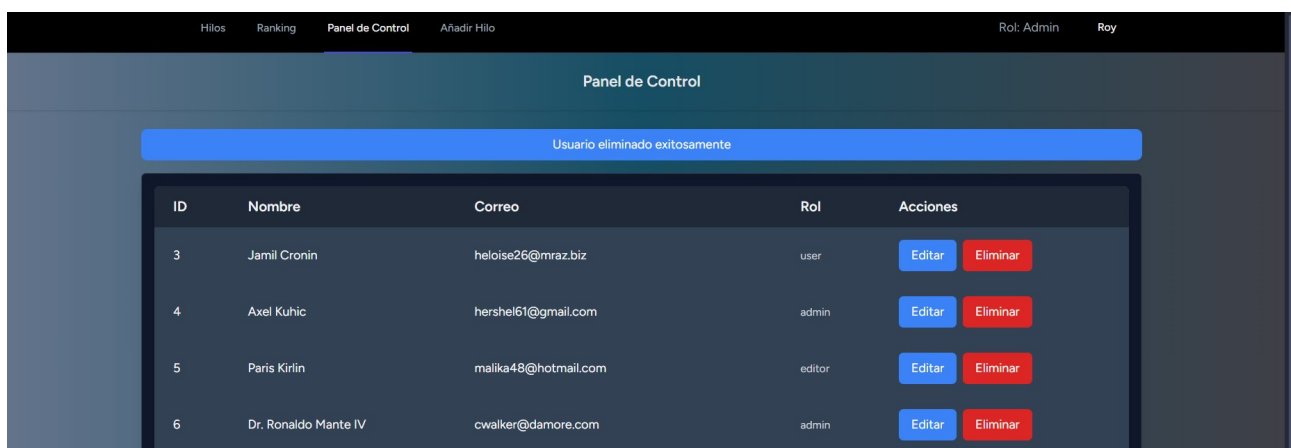
```

Admin:

Para controlar que el usuario es administrador lo hacemos con Spatie haciendo uso de los permisos. Estos permisos se encuentran asignados a el rol administrador y con el bloque `@can` de blade controlamos mostrar o no según que permisos tenga el usuario en este caso habiamos definido los permisos "admin.del.user", "admin.del.post" y "admin.edit.rol".

Los pasos para admin.del.user es se muestra o no dependiendo del permiso en este caso lo tenemos y si hacemos click se borrará este usuario. Podremos ver el resultado en la base de datos.

```
@can('admin.del.user')
<form action="{{ route('admin.del.user', $user) }}" method="POST" class="inline">
  @csrf
  @method('DELETE')
  <button type="submit"
    class="px-4 py-2 rounded-md bg-red-600 text-white hover:bg-red-800 transition duration-300">
    Eliminar
  </button>
</form>
@endcan
```



	id	name	email	password	img_perfil	created_at
☐ Editar ☐ Copiar ☐ Borrar	3	Jamil Cronin	heloise26@mraz.biz	\$2y\$12\$LBV776Njo6ovSsPR9nE9g.xVO.gEXcpobBmkFA6KBCn...	avatar3.png	2025-02-25 15:09:08
☐ Editar ☐ Copiar ☐ Borrar	4	Axel Kuhic	hershel61@gmail.com	\$2y\$12\$CepMBVH.W8mYg.cLEJ5q.z.QJfloi1QxQUgOSzNGo...	avatar1.png	2025-02-25 15:09:08
☐ Editar ☐ Copiar ☐ Borrar	5	Paris Kirlin	malika48@hotmail.com	\$2y\$12\$jGtE3q\$ZAZ.JxZ7TW.InErUOhqmwY/E9l3C3HSFqxtTIC...	avatar4.png	2025-02-25 15:09:08
☐ Editar ☐ Copiar ☐ Borrar	6	Dr. Ronaldo Mante IV	cwalker@damore.com	\$2y\$12\$6DaA3eGg456DtpqlyOXZGCOunmOBykqK5AVSQ38...	avatar4.png	2025-02-25 15:09:08
☐ Editar ☐ Copiar ☐ Borrar	7	Arturo Kunze	mertie.luetgen@wisozk.com	\$2y\$12\$fstWpql70FAZVsKCxb.D.PXjl1S0Xysz7nBgRG1LXY...	avatar5.png	2025-02-25 15:09:08
☐ Editar ☐ Copiar ☐ Borrar	8	Shana Hodkiewicz	tsporer@gmail.com	\$2y\$12\$GzUjlxKL68qmkCfHfc1weMDSVa0annOIMXg6dGvNolj...	avatar3.png	2025-02-25 15:09:08
☐ Editar ☐ Copiar ☐ Borrar	9	Dr. Tyrel Abshire Sr.	christophe.erdman@hotmail.com	\$2y\$12\$LatK5wGYHOH27vMnQ7su1oLskzmVetzdTOTCAf8w...	avatar5.png	2025-02-25 15:09:08
☐ Editar ☐ Copiar ☐ Borrar	10	Price Weber	gusikowski.brooks@turner.com	\$2y\$12\$K/LdIWVJR2ozSj9GE5mOatyYfyJ5aWxc8lkyvyE/W...	avatar2.png	2025-02-25 15:09:08

Para admin.del.post es lo mismo si tiene el permiso el usuario o no se mostrará un botón en la parte inferior derecha del post para eliminar que si hacemos click en el borrará dicho post y los likes que tenga. Haremos una prueba para ver que en la base de datos se elimino.

```

@can('admin.del.post')
    <form action="{{ route('admin.del.post', $post) }}" method="POST">
        @csrf
        <button type="submit"
            class="bg-red-600 p-2 rounded-lg hover:bg-red-800 transition flex items-center space-x-1">
            <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" fill="#FFF"
                viewBox="0 0 24 24">
                <path
                    d="M9 3V2a1 1 0 0 1 1-1h4a1 1 0 0 1 1 1v1h5a1 1 0 1 1 0 2h-1v15a3 3 0 0 1-3 3H7a3 3 0 0 1-3-3V5H3a1"
                />
            </svg>
            <span class="text-white text-sm">Eliminar</span>
        </button>
    </form>
@endcan

```

En el controlador recibimos su id de post y buscamos si existe y si existe lo eliminamos.

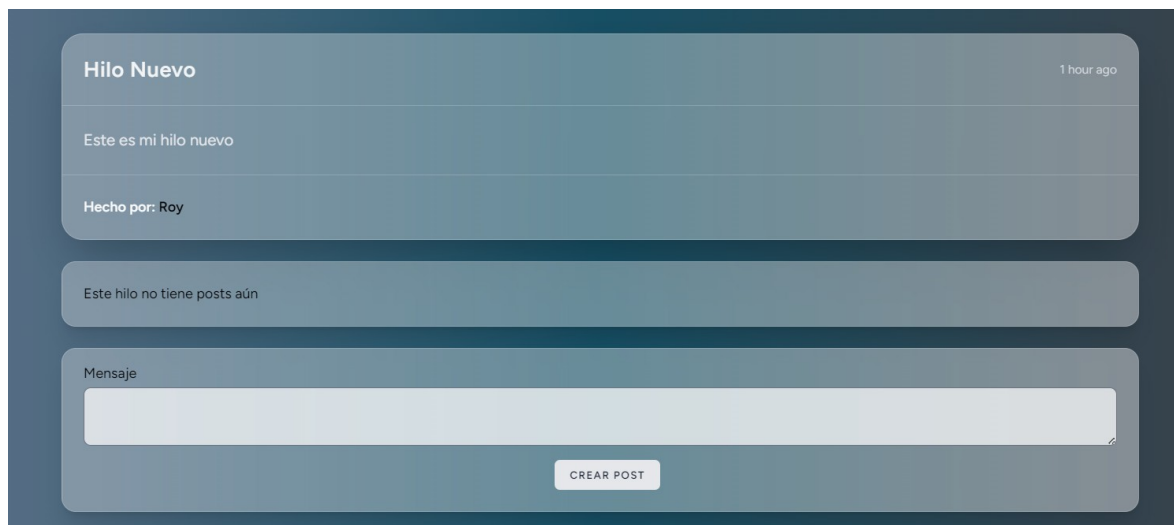
```

public function destroy(Request $request, $post)
{
    if (!$request->isMethod('post')) {
        return redirect()->back();
    }

    $post = Post::findOrFail($post)->delete();
    return redirect()->back();
}

```

Nosotros eliminaremos de prueba el post que creamos anteriormente.



Se habrá eliminado el post correctamente.

Y para admin.edit.rol es lo mismo solo que en formulario controlamos si el usuario tiene permiso para cambiar rol y le mostramos el select con los roles disponibles y en el FormRequest comprobamos igualmente si el usuario tiene permisos para cambiar el rol para que valide el campo si es administrador.


```
@can('admin.edit.rol')
<div class="mt-4">
    <x-input-label for="rol" :value="__('Rol')" />
    <select name="rol" id="rol" class="block mt-1 w-full">
        @foreach ($roles as $rol)
            <option value="{{ $rol }}" {{ $user->hasRole($rol) ? 'selected' : '' }}>
                {{ $rol }}</option>
        @endforeach
    </select>
    <x-input-error :messages="$errors->get('rol')" class="mt-2" />
</div>
@endcan
```

```
class EditRequest extends FormRequest
{
    /**
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        $userId = $this->route('user');

        return [
            'name' => ['required', 'string', 'min:3', 'max:20', 'unique:users,name,' . $userId],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users,email,' . $userId],
            'img_perfil' => ['image', 'mimes:jpeg,png,jpg,webp', 'max:10240'],
            'password' => ['nullable', Rules\Password::defaults()],
        ];

        if (auth()->user->hasRole('admin')) {
            $rules['rol'] = ['required', 'string', 'in:admin,editor,user'];
        }
    }
}
```

Si cambiamos el usuario podremos ver como en la base de datos cambia nuestro rol.

Editar Usuario: Roy

Nombre
Roy

Correo Electronico
roymodificado063@g.educaand.es

Imagen
Seleccionar archivo Ningún archivo seleccionado

Rol
admin
admin
editor
user

MODIFICAR DATOS

En este caso lo cambiaremos a editor y podremos ver que se actualizo correctamente el rol. (El rol id 2 corresponde al rol de editor).

				role_id	model_type	model_id
☐		Editar		Copiar		Borrar
				1	App\Models\User	4
☐		Editar		Copiar		Borrar
				1	App\Models\User	6
☐		Editar		Copiar		Borrar
				1	App\Models\User	8
☐		Editar		Copiar		Borrar
				1	App\Models\User	10
☐		Editar		Copiar		Borrar
				1	App\Models\User	13
☐		Editar		Copiar		Borrar
				1	App\Models\User	15
☐		Editar		Copiar		Borrar
				2	App\Models\User	5
☐		Editar		Copiar		Borrar
				2	App\Models\User	17
☐		Editar		Copiar		Borrar
				2	App\Models\User	19
☐		Editar		Copiar		Borrar
				2	App\Models\User	21
☐		Editar		Copiar		Borrar
				3	App\Models\User	3
☐		Editar		Copiar		Borrar
				3	App\Models\User	7
☐		Editar		Copiar		Borrar
				3	App\Models\User	9
☐		Editar		Copiar		Borrar
				3	App\Models\User	11

Nuestro usuario

Imagen y Rutas:

Para las imagenes creamos un disco nuevo virtual que se llamara users, lo definimos en el fichero de configuración de laravel filesystems.php

```
return [  
    'disks' => [  
        'local' => [  
            'driver' => 'local',  
            'root' => storage_path('app/private'),  
            'serve' => true,  
            'throw' => false,  
            'report' => false,  
        ],  
        'users' => [  
            'driver' => 'local',  
            'root' => storage_path('app/users'),  
            'url' => env('APP_URL').'/storage',  
            'visibility' => 'public',  
            'serve' => true,  
            'throw' => false,  
            'report' => false,  
        ],  
        'public' => [  
            'driver' => 'local',  
            'root' => storage_path('app/public'),  
            'url' => env('APP_URL').'/storage',  
            'visibility' => 'public',  
            'throw' => false,  
            'report' => false,  
        ],  
    ],  
]
```

Luego creamos un controlador para la imagen que nos permitirá cargar la imagen de nuestro disco virtual y mostrarla en la pagina.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;

class ImageController extends Controller
{
    public function getImage($filename)
    {
        $file = Storage::disk('users')->get('profile/' . $filename);
        return response($file, 200);
    }
}
```

Y cuando solicitemos mediante una ruta la imagen el servidor nos dará con respuesta la imagen.

```
@if (Route::has('login'))
<nav class="-mx-3 flex flex-1 justify-end">
    <div class="text-gray-400 dark:text-gray-400 flex items-center">Rol:
        {{ ucfirst(Auth::user()->getRoleNames()->first()) }}</div>
    <button
        class="inline-flex items-center px-3 py-2 border border-transparent text-sm leading-4 font-medium rounded-md text-gray-500 dark:text-gray-400 hover:text-gray-700 dark:hover:text-white"
    >
        <div class="ms-1 flex items-center">
            
        </div>
        <div class="ml-2 dark:text-gray-100">{{ Auth::user()->name }}</div>
    </button>
</nav>
@endif
```

Para cambiar actualizar las imágenes o para registrarlas en el controlador utilizamos `Storage::disk('users')` para especificar el disco y el método `put` para guardar la imagen o `delete` para borrarla. En este caso para actualizar la imagen de un usuario borramos la anterior y guardamos la nueva, cuando esta guardada guardamos el nombre de la img y se la pasamos al usuario y le actualizamos ese campo en la base de datos.

```

class UserController extends Controller
{
    public function edit($user)
    {

    }

    public function update(EditRequest $request, $user)
    {
        $request->validated();

        $user = User::findOrFail($user);

        $imgname = null;
        if ($request->hasFile('img_perfil')) {
            Storage::disk('users')->delete('profile/', $user->img_perfil);
            Storage::disk('users')->put('profile/', $request->file('img_perfil'));

            $imgname = $request->file('img_perfil')->hashName();
        } else {
            $user->img_perfil = $user->img_perfil;
        }

        $user->update([
            'name' => $request->name,
            'email' => $request->email,
            'password' => $request->password ? Hash::make($request->password) : $user->password,
            'img_perfil' => $imgname ? $imgname : $user->img_perfil
        ]);

        if (auth()->guard('web')->user()->roles[0]->name == 'admin') {
            $user->syncRoles($request->rol);
        }

        return redirect('/control')->with('status', 'Usuario actualizado exitosamente');
    }
}

```

Para crear un usuario es practicamente lo mismo.

```

class RegisteredUserController extends Controller
{
    /**
     * Handle an incoming registration request.
     *
     * @throws \Illuminate\Validation\ValidationException
     */
    public function store(RegisterRequest $request): RedirectResponse
    {
        $request->validated();
        Storage::disk('users')->put('profile/', $request->file('img_perfil'));

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
            'img_perfil' => $request->file('img_perfil')->hashName(),
        ]->assignRole("user");

        event(new Registered($user));

        Auth::login($user);

        return redirect()->back();
    }
}

```

Por ultimo en el fichero de rutas agrupamos las rutas por roles para que los usuarios con acceso no permitido no puedan acceder a estas rutas.

Usamos un middleware por cada grupo para identificar el acceso que deben tener los jugadores y no permitir que accedan a lugares que no deseamos que entren, por ejemplo tenemos el middleware que comprueba que los jugadores deben estar autenticados y tener el rol admin o editor, así como hay otro para los admins que son las rutas de eliminar post y eliminar user y por ultimo las rutas generales de funcionalidad de la app que es para crear post, crear hilo, editar sus datos, dar likes, ver imágenes, ver los hilos y los posts.

```
6 use App\Http\Controllers\ProfileController;
7 use App\Http\Controllers\UserController;
8 use Illuminate\Support\Facades\Route;
9
10 Route::get('/', [HiloController::class, 'index'])->name('home');
11 Route::get('hilo/{id}/posts', [PostController::class, 'verPost'])->name('verPost');
12 Route::get('images/{filename}', [ImageController::class, 'getImage'])->name('images.get');
13
14 Route::get('/ranking', [UserController::class, 'ranking'])->name('ranking');
15
16 // Grupo para usuarios con el rol de admin o editor
17 Route::middleware(['auth', 'role:admin|editor'])->group(function () {
18     Route::get('/control', [UserController::class, 'panel'])->name('ver.users');
19     Route::get('/usuarios/{user}/edit', [UserController::class, 'edit'])->name('edit.users');
20     Route::post('/usuarios/{user}/update', [UserController::class, 'update'])->name('update.users');
21 });
22
23 // Grupo para usuarios autenticados
24 Route::middleware('auth')->group(function () {
25     Route::get('/user/hilo/add', [HiloController::class, 'addHilo'])->name('user.add.hilo');
26     Route::post('/user/hilo/create', [HiloController::class, 'createHilo'])->name('user.create.hilo');
27     Route::post('/user/post/create/{post}', [PostController::class, 'createPost'])->name('user.create.post');
28     Route::post('/user/post/like/{post}', [PostController::class, 'like'])->name('user.like.post');
29 });
30
31 // Grupo para usuarios con el rol de admin
32 Route::middleware(['auth', 'role:admin'])->group(function () {
33     Route::delete('/control/delete/{user}', [UserController::class, 'destroy'])->name('admin.del.user');
34     Route::delete('/post/delete/{post}', [PostController::class, 'destroy'])->name('admin.del.post');
35 });
36
37 // Grupo para usuarios autenticados (Control de perfil)
38 Route::middleware('auth')->group(function () {
39     Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
40     Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
41     Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
42 });
```

El proyecto funciona correctamente sin problemas y se cumplen todos los requisitos de la tarea incluidos un poco de diseño y estilo para cambiar los estilos por defectos que trae Laravel.